The Essential of ML: Chapter 3

From Fitting to Generalization, cut down to what the twenty review questions and Tutorial 1 test: the gap, capacity, bias and variance, penalties, selection, honest splits and metrics

Fundamentals of Machine Learning

2026-10-30

How to read this document

This is Chapter 3 reduced to what the professor's *Review questions, Lecture 3* (twenty questions) and Problems 4 and 5 of *Tutorial 1* test. Six sections in concept order: every symbol is defined before use, every claim carries its two-line derivation or worked number, and each section closes with a *Tutorial drill* box in the professor's own style ("Explain why…", "Derive…", "Compute…"), answered as an expert would: claim, mechanism, number, consequence. Review questions by section: Q1–4 in §1, Q5–7 in §2, Q8–10 in §3, Q11–13 in §4, Q14–16 in §5, Q17–20 in §6. Left out on purpose: the Lab 1 walkthrough, the slide errata, and optimisation error (Lecture 6). Notation: $\widehat{R}$ is an average over a sample, R an expectation over the world, $\mathcal{D}$ a training set, $\mathcal{H}$ a hypothesis space, $\hat{f}_{\mathcal{D}}$ the model a fixed procedure returns when fed $\mathcal{D}$.

1 Fitting Is Not Learning

Empirical risk, population risk, generalization gap

For a loss ℓ and a model f, the **empirical risk** on a dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ and the **population risk** are

$$\widehat{R}_{\mathcal{D}}(f) = \frac{1}{N}\sum_{i=1}^{N} \ell(y^{(i)}, f(x^{(i)})), \qquad R(f) = \mathbb{E}_{(X,Y)\sim\mathbb{P}}[\ell(Y, f(X))].$$

Same loss, two populations of examples: the ones you have, and the ones you will meet. For the model $\hat{f}_{\mathcal{D}}$ fitted on $\mathcal{D}$, the **generalization gap** is

$$G(\hat{f}_{\mathcal{D}}) = R(\hat{f}_{\mathcal{D}}) - \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}).$$

Only $\widehat{R}_{\mathcal{D}}$ is computable during training: it needs the data and the model. R is an expectation under the unknown distribution $\mathbb{P}$, and G contains R. A held-out set gives an *estimate* of R, never R itself.

Empirical risk minimisation (ERM) picks $\hat{f}_{\mathcal{D}} \in \arg\min_{f\in\mathcal{H}} \widehat{R}_{\mathcal{D}}(f)$. That is a *fitting* problem; the purpose of learning is R, and a fit "generalizes" when G is small. A spam filter with 2% errors on training mail and 9% on next month's has gap 0.07: the training score flattered it by seven points.

Why training performance is optimistic (Review Lec 3, Q2)

The training data play two roles: they first choose *which* model is selected, then they grade it, $\mathcal{D} \to \hat{f}_{\mathcal{D}} \to \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$. The fit can therefore exploit accidental regularities of $\mathcal{D}$ (noise, coincidences) that will not recur. Formally, let $f^*_{\mathcal{H}}$ minimise R over $\mathcal{H}$ (a fixed function, independent of the sample). Then

$$\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \;\le\; R(f^*_{\mathcal{H}}) \;\le\; \mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})], \qquad \text{hence} \qquad \mathbb{E}_{\mathcal{D}}[G(\hat{f}_{\mathcal{D}})] \ge 0.$$

Left: for every sample $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ because $\hat{f}_{\mathcal{D}}$ minimises $\widehat{R}_{\mathcal{D}}$; take expectations and use that for a *fixed* f the empirical risk is unbiased, $\mathbb{E}_{\mathcal{D}}\widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}}) = R(f^*_{\mathcal{H}})$. *Right*: pointwise, $R(\hat{f}_{\mathcal{D}}) \ge \min_{\mathcal{H}} R$. On average the training score is below even the best risk any member of $\mathcal{H}$ can reach.

Key insight. Low training loss is *necessary* evidence that optimisation worked and *incomplete* evidence that learning worked. Everything else in this chapter exists to measure a model on data that did not choose it.

Stable and spurious cues. If training cows stand on grass and camels on sand, both *shape* and *background* predict every training label; training risk cannot tell them apart because both score zero. A cow on sand does. A spurious cue yields zero training loss and a large gap, so dataset composition is *part of the learning method*.

Inductive bias (Review Lec 3, Q4)

A finite sample is compatible with infinitely many functions that agree on the sample and differ elsewhere. Learning therefore needs **inductive bias**: assumptions that make some solutions more plausible than others beyond the observed examples. It enters through four channels: (1) the *data and representation* (which cues exist at all); (2) the *hypothesis space* $\mathcal{H}$ (which relationships can be expressed); (3) *regularisation* (which fitting solutions are preferred); (4) the *optimisation algorithm* (which minimiser it reaches when several tie). Here "bias" means a preference among solutions; it is broader than the statistical bias of Section 3. Generalization succeeds exactly when these assumptions remain true at deployment.

Tutorial drill

Review Lec 3, Q1. *Distinguish empirical risk, population risk and the generalization gap. Why is only one observable?*
Three objects: $\widehat{R}_{\mathcal{D}}(f)$ is an average of the loss over the N examples in hand; $R(f)$ is the expectation of the same loss over the deployment distribution; $G(\hat{f}_{\mathcal{D}}) = R(\hat{f}_{\mathcal{D}}) - \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$ compares the two *for the same fitted model*. Only $\widehat{R}_{\mathcal{D}}$ can be computed: R needs $\mathbb{P}$, which is unknown, and G needs R. Consequence: every "performance" we quote is an estimate of R from data the model never saw, and its quality depends entirely on how that data was held out (Section 6).
Review Lec 3, Q3. *One stable and one spurious cue in a security application; how could data collection reveal the difference?*
Stable: distinct destination ports per minute with a high connection-failure fraction, a property of what a scan *is*. Spurious: the testbed attacker's subnet, a TTL specific to the attack VM, or time of day if attacks were only simulated at night; these predict labels only because of how the data were produced. Collection reveals the difference by *breaking the correlation*: launch attacks from other hosts, OSes, subnets and times; collect benign traffic at the attack times; evaluate on grouped or temporal splits. A stable cue survives, a spurious cue collapses to chance.
New. *A model scores* 0.02 *on training mail and* 0.09 *on next month's mail. Give the gap. Can a gap be negative?*
$G \approx 0.09 - 0.02 = 0.07$ (the 0.09 is itself an estimate of R). For one sample G can be negative,

since $\widehat{R}_{\mathcal{D}}$ is random; the proposition fixes only the *sign of its expectation*. A repeatedly negative gap is a symptom (a leak making the test set easier than training, or a favourable shift), not a triumph.

2 Capacity: Flexibility Always Looks Good on Training Data

Capacity and nested families

The **capacity** of $\mathcal{H} = \{f_\theta : \theta \in \Theta\}$ describes how varied the functions in it can be. It is *not* parameter count alone: structure, constraints on θ, regularisation, the optimiser and the amount of data all matter. Capacity dials: polynomial degree, tree depth, network width, and the neighbourhood size k of kNN. Families are **nested** when $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \ldots$: every function of a smaller family remains available in a larger one.

Minimum training error is monotone in capacity (Review Lec 3, Q5)

For nested families $\mathcal{H}_m \subseteq \mathcal{H}_{m+1}$,

$$\min_{f \in \mathcal{H}_{m+1}} \widehat{R}_{\text{train}}(f) \;\leq\; \min_{f \in \mathcal{H}_m} \widehat{R}_{\text{train}}(f).$$

Proof in one line: the minimiser over $\mathcal{H}_m$ is itself a candidate in $\mathcal{H}_{m+1}$, so the minimum over the larger set is at most its value; a minimum over a superset can never be larger. (Assumes the optimiser actually finds the minimum.) Consequence: training error *alone* systematically favours the richest family, so it can never be used to choose capacity; it says nothing about R, because the extra functions can fit stable structure *or* accidental variation.

Underfitting and overfitting, read from two numbers (Review Lec 3, Q6)

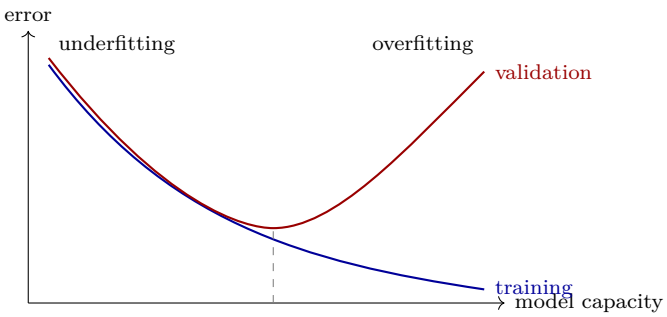
Underfitting: the family is too restrictive; training *and* validation error are both high and close together. **Overfitting**: the model is too sensitive to the particular sample; training error is low and validation error substantially higher (large gap). The diagnostic is the *pair* and the gap between them, never one number alone. "Complicated" is not the criterion; the criterion is whether effective flexibility matches the evidence available.

Three polynomials on four points, judged on a fifth

Fit $(0,1),(1,3),(2,2),(3,4)$ by least squares with degrees 0, 1, 3; training risks 1.25, 0.45, 0. A validation point $(4,5)$ arrives. Degree 0 predicts 2.5: error $(5-2.5)^2 = 6.25$. Degree 1, the line $0.8x+1.3$, predicts 4.5: error 0.25. Degree 3, the interpolating cubic $1+5.5x-4.5x^2+x^3$ (check: $f(1)=3$, $f(2)=2$, $f(3)=4$), predicts $1+22-72+64=15$: error 100.

degree	0	1	3
training risk	1.25	0.45	0.00
validation error at $x=4$	6.25	0.25	100

The constant underfits (bad everywhere); the cubic overfits (perfect on the points that shaped it, absurd one step outside, having bent itself around the noise). The line, worse than the cubic on training data, is the only one worth deploying.



kNN and the direction of the dial (Review Lec 3, Q7). The kNN prediction at x averages (or votes over) the k nearest training labels. With $k = 1$ each training point rules its own neighbourhood, the rule follows every observation and the training error is 0 (barring duplicate inputs with different labels). Increasing k averages over more points, so the fitted function changes more slowly and can no longer track individual observations; at $k = N$ it is the global mean or majority class everywhere, a constant. Roughly, kNN has about N/k independent "pieces", so *decreasing* k increases capacity; students routinely get this backwards.

Key insight. Capacity is relative to sample size. A family unstable on $N = 100$ may be well constrained by $N = 100{,}000$: more observations leave fewer functions that fit the sample while behaving arbitrarily elsewhere. "Too complex" is a property of $\mathcal{H}$ *given the evidence*, never of $\mathcal{H}$ alone.

Tutorial drill

Tutorial 1, Q5(a)–(c). *Nested* $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \mathcal{H}_3$*; training errors* $0.20, 0.08, 0.00$*; validation errors* $0.22, 0.11, 0.19$*. Why can training error not increase? Which family? Diagnose. After much more data the validation error of* $\mathcal{H}_3$ *falls to* 0.10*: why can capacity not be judged without* N*?*
(a) Every function of $\mathcal{H}_1$ is in $\mathcal{H}_3$, so $\min_{\mathcal{H}_3} \widehat{R}_{\text{train}} \leq \min_{\mathcal{H}_2} \widehat{R}_{\text{train}} \leq \min_{\mathcal{H}_1} \widehat{R}_{\text{train}}$: the larger set contains the smaller set's minimiser as a candidate. (b) Select $\mathcal{H}_2$ (lowest validation error 0.11, gap 0.03). $\mathcal{H}_1$: both errors high and close (0.20/0.22), underfitting, high approximation error. $\mathcal{H}_3$: interpolates the sample (0.00) with gap 0.19, excessive capacity *for this sample*, high estimation variance. (c) Variance shrinks with N: with more data fewer functions in $\mathcal{H}_3$ fit the sample while misbehaving elsewhere, so the fitted function stabilises and $\mathcal{H}_3$ becomes the best ($0.10 < 0.11$). The same family that overfitted a small sample is well constrained by a large one; the right capacity grows with N.
New. *Give a counterexample to "the model with the lower training error is the better model."* The polynomial table: degree 3 has training risk 0, degree 1 has 0.45, yet on the new point degree 1 errs by 0.25 and degree 3 by 100. The nested-family proposition guarantees training error falls with degree, so it carries no information about which degree generalizes; capacity is chosen on a held-out set, never on the training score. (Review Lec 3, Q7 is answered in the kNN paragraph above: the dial is $1/k$, from $k = 1$, training error 0, to $k = N$, a constant.)

3 Bias and Variance: Making "Sensitive to the Sample" a Number

Bias and variance at a fixed input (Review Lec 3, Q8)

Imagine drawing training sets $\mathcal{D}_1, \mathcal{D}_2, \ldots$ from the same population and running the *same* learning procedure on each. At a fixed input x this gives a cloud of predictions $\hat{f}_{\mathcal{D}_1}(x), \hat{f}_{\mathcal{D}_2}(x), \ldots$. For squared-error regression let $f^*(x) = \mathbb{E}[Y \mid X = x]$ be the optimal regression function and

$\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$ the **average prediction** across training sets. Then

$$\text{Bias}(x) = \bar{f}(x) - f^*(x), \qquad \text{Var}(x) = \mathbb{E}_{\mathcal{D}}\Big[\big(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x)\big)^2\Big].$$

Bias asks whether the *centre* of the cloud is off the truth in a consistent direction; variance asks how *wide* the cloud is around its own centre. Neither can be computed from one dataset: they describe the *procedure*, not a fitted model. A rule that always predicts "1 million" has zero variance and possibly large bias; a very flexible one may have tiny bias and huge variance.

Restrictive families and poor features create bias, which more data does not remove if the family cannot express f^*. High capacity, small samples, irrelevant variables and unstable feature selection create variance, which more representative data reduce.

The three-term decomposition, derived (Review Lec 3, Q9)

Assume $Y = f^*(X) + \varepsilon$ with $\mathbb{E}[\varepsilon \mid X] = 0$, $\text{Var}(\varepsilon \mid X = x) = \sigma^2(x)$, and the training sample $\mathcal{D}$ independent of the test pair (x, Y). Then at every fixed x

$$\mathbb{E}_{\mathcal{D},\,Y|x}\Big[\big(Y - \hat{f}_{\mathcal{D}}(x)\big)^2\Big] = \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x).$$

Step 1 (split off the noise). $Y - \hat{f}_{\mathcal{D}}(x) = \varepsilon + (f^*(x) - \hat{f}_{\mathcal{D}}(x))$. Square: $\varepsilon^2 + 2\varepsilon\,(f^* - \hat{f}_{\mathcal{D}}) + (f^* - \hat{f}_{\mathcal{D}})^2$. Take expectations: $\mathbb{E}[\varepsilon^2 \mid x] = \sigma^2(x)$; the cross term factorises because ε is independent of $\mathcal{D}$ and has mean zero, $\mathbb{E}[\varepsilon] \cdot \mathbb{E}_{\mathcal{D}}[f^* - \hat{f}_{\mathcal{D}}] = 0$.
Step 2 (split around the average prediction). $f^*(x) - \hat{f}_{\mathcal{D}}(x) = (f^*(x) - \bar{f}(x)) + (\bar{f}(x) - \hat{f}_{\mathcal{D}}(x))$. The first bracket is a constant, $-\text{Bias}(x)$, whose square is $\text{Bias}(x)^2$; the second has $\mathbb{E}_{\mathcal{D}}$-mean zero by definition of $\bar{f}$, so the cross term $2(f^* - \bar{f})\,\mathbb{E}_{\mathcal{D}}[\bar{f} - \hat{f}_{\mathcal{D}}]$ vanishes; its square averages to $\text{Var}(x)$. Add the pieces.
Twice the same move: add and subtract a mean, square, and watch the cross term die because a mean-zero quantity multiplies a constant.

The three terms at one input: $\sigma^2(x)$ is **irreducible noise**, the error even f^* makes because x does not determine Y; $\text{Bias}(x)^2$ is **systematic error**, which repeating the experiment never averages away; $\text{Var}(x)$ is **sample sensitivity**, the part a different training set would change. Increasing capacity typically lowers bias and raises variance, which produces the U-shaped validation curve of Section 2. This is a diagnostic principle, not a law.

Bias and variance computed exactly: the shrunk mean

No input: $Y = \mu + \varepsilon$, $\text{Var}(\varepsilon) = \sigma^2$, so $f^* = \mu$; the training set is N draws. *Sample mean* $\hat{f} = \bar{y}$: $\mathbb{E}_{\mathcal{D}}\hat{f} = \mu$, bias 0, variance σ^2/N. *Ignore the data* $\hat{f} = 0$: bias $-\mu$, variance 0. *Shrunk mean* $\hat{f} = \frac{1}{2}\bar{y}$: bias $-\mu/2$, variance $\sigma^2/(4N)$. With $\sigma^2 = 1$, $N = 4$, bias^2+variance is

	sample mean	ignore data	shrunk mean
$\mu = 1$	$0 + 0.25 = 0.25$	$1 + 0 = 1$	$0.25 + 0.0625 = 0.3125$
$\mu = 0.2$	$0 + 0.25 = 0.25$	$0.04 + 0 = 0.04$	$0.01 + 0.0625 = 0.0725$

Far from zero the unbiased procedure wins; near zero both biased procedures beat it by trading a little systematic error for a large drop in variance. Regularisation (Section 4) is this trade, made tunable. Predicting a *new* Y adds $\sigma^2 = 1$ to every cell: the floor nobody beats.

Excess error over the Bayes risk has three sources: *approximation* ($\mathcal{H}$ cannot express a good function), *estimation* (finite data do not identify the best member of $\mathcal{H}$), *optimisation* (the algorithm does not reach its own optimum). Capacity trades the first against the second. The classical picture assumes the trade is monotone; it is not always.

Double descent and the interpolation threshold (Review Lec 3, Q10)

The **interpolation threshold** is the smallest capacity at which the model can fit the training sample exactly (training error 0). In modern over-parameterised models the validation error may rise sharply as capacity approaches it, then *fall again* beyond it: **double descent**. The second descent depends on regularisation, the optimiser, the data geometry and *which* interpolating solution is selected; it does not occur in every problem. The same shape can appear over *training time* at fixed parameter count (epoch-wise double descent). The classical U-shape remains the right first model and a valid diagnostic in the under-parameterised regime, but validation behaviour must be *examined*, not assumed.

Tutorial drill

Review Lec 3, Q9. *Identify the three terms in the squared-error decomposition. Which cannot be removed even by the optimal predictor?*
$\mathbb{E}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x)$: noise, squared bias, variance. The noise $\sigma^2(x) = \text{Var}(Y \mid X = x)$ is irreducible: plugging $\hat{f} = f^*$ into the identity gives bias 0, variance 0 and leaves $\sigma^2(x)$, because X does not determine Y. Example: Bias $= 0.3$, Var $= 0.05$, $\sigma^2 = 0.1$ gives $0.1 + 0.09 + 0.05 = 0.24$, of which 0.1 no procedure can touch. Consequence: a model can be "perfect" and still wrong on a noisy world; compare against the noise floor, not against zero.
Review Lec 3, Q10. *What is the interpolation threshold? Why is the U-curve useful but not universal?*
The smallest capacity at which training error reaches 0. Just there the fit must pass through every noisy point, so variance peaks; beyond it the optimiser chooses among many interpolants and, with regularisation, may pick smooth ones, so validation error can fall again. The U-shape is correct in the under-parameterised regime and the right first diagnostic; it is not a law because the second descent occurs in some (not all) over-parameterised models, and over epochs. So plot the curve rather than assume it, and keep checkpoints when early stopping.
New (Math set 3, Problem 3.2 condensed). *kNN regression at $x = 0$ with $f^*(x) = x^2$, fixed inputs $\pm 0.1, \ldots, \pm 0.5$, noise variance 0.01. Compute bias, variance and bias^2+variance for $k = 2$ and $k = 10$.*
$\hat{f}_k(0) = \frac{1}{k}\sum_{N_k} y_i$, so $\text{Bias}(0) = \frac{1}{k}\sum_{N_k} x_i^2 - 0$ and $\text{Var}(0) = \sigma^2/k$ (average of k independent noises). $k = 2$: neighbours ± 0.1, bias 0.01, variance 0.005, total 0.0051. $k = 10$: bias $2(0.01 + 0.04 + 0.09 + 0.16 + 0.25)/10 = 0.11$, variance 0.001, total 0.0131. (The minimum is at $k = 4$: $0.000625 + 0.0025 = 0.0031$.) Small k is noisy, large k averages in points where x^2 is far from 0: the U-shape with $1/k$ as capacity, in numbers.

4 Controlling Effective Capacity: Penalties and Early Stopping

Regularised ERM

Given a **penalty** $\Omega : \Theta \to [0, \infty)$ measuring how "complicated" θ is, and a strength $\lambda \geq 0$,

$$\hat{\theta}_\lambda \in \arg\min_\theta \left[\widehat{R}_{\text{train}}(\theta) + \lambda\,\Omega(\theta)\right], \qquad \Omega_2(\theta) = \|\theta\|_2^2 = \sum_j \theta_j^2 \ (L_2), \quad \Omega_1(\theta) = \|\theta\|_1 = \sum_j |\theta_j| \ (L_1).$$

λ prices a unit of penalty in units of training loss: $\lambda = 0$ is plain ERM; large λ makes any sizeable coefficient too expensive. For $\theta = (3, -4)$: $\Omega_2 = 25$, $\Omega_1 = 7$. Standardise features first (otherwise a feature in millimetres is penalised far more than the same feature in metres) and do not penalise the intercept.

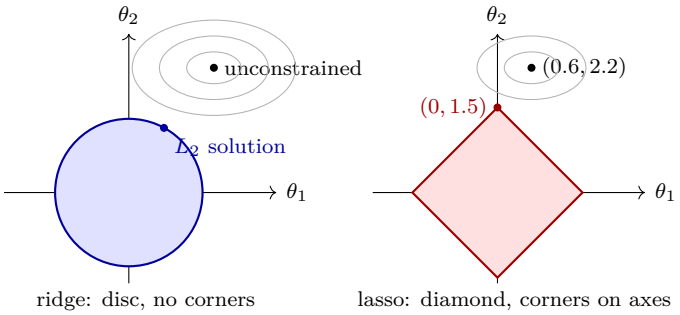
A penalty is a soft constraint: fewer functions, same parameter count (Review Lec 3, Q11)

Let $\hat{\theta}_\lambda$ solve the penalised problem and put $c = \Omega(\hat{\theta}_\lambda)$. Then $\hat{\theta}_\lambda$ also solves $\min_\theta \widehat{R}_{\text{train}}(\theta)$ subject to $\Omega(\theta) \le c$. *Proof*: for any feasible θ ($\Omega(\theta) \le c$), optimality gives $\widehat{R}(\hat{\theta}_\lambda) + \lambda c \le \widehat{R}(\theta) + \lambda\Omega(\theta) \le \widehat{R}(\theta) + \lambda c$, hence $\widehat{R}(\hat{\theta}_\lambda) \le \widehat{R}(\theta)$. So regularised ERM fits within $\mathcal{H}_c = \{f_\theta : \Omega(\theta) \le c\}$, and these families are nested, $c_1 < c_2 \Rightarrow \mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$, with larger λ giving smaller c. The parameter vector keeps its length; fewer of its values can win the fitting contest. Corollary: training risk is minimised at $\lambda = 0$ (Section 2's monotonicity on the nested $\mathcal{H}_c$), so λ *must* be chosen on validation data.

Ridge shrinks, lasso thresholds: the one-variable calculations

Ridge. For $f_w(x) = wx$ under squared loss, $J(w) = \frac{1}{N}\sum_i (y^{(i)} - wx^{(i)})^2 + \lambda w^2$ is a convex quadratic; $J'(w) = 0$ gives $\hat{w}_\lambda = \dfrac{\sum_i x^{(i)} y^{(i)}}{\sum_i (x^{(i)})^2 + N\lambda}$, the least-squares slope times $\dfrac{\sum x^2}{\sum x^2 + N\lambda} \in (0, 1]$: shrunk toward 0, smoothly, never exactly 0. With $\sum x^2 = 14$, $\sum xy = 19$, $N = 4$: $\lambda = 0 \to 1.357$; $\lambda = 1 \to 1.056$; $\lambda = 10 \to 0.352$.

Lasso. Minimise $\frac{1}{2}(w - z)^2 + \lambda|w|$, z being where the unpenalised fit would put the coefficient. For $w > 0$ the derivative $w - z + \lambda$ vanishes at $w = z - \lambda$, valid only if $z > \lambda$; for $w < 0$, $w = z + \lambda$, valid only if $z < -\lambda$; if $|z| \le \lambda$ neither is admissible and the minimum sits at the kink, $w = 0$. Together $\hat{w} = \text{sign}(z)\max(|z| - \lambda, 0)$, *soft thresholding*: with $\lambda = 0.5$, $2.2 \to 1.7$, $0.4 \to 0$, $-3 \to -2.5$. The L_2 analogue $\frac{1}{2}(w - z)^2 + \lambda w^2$ gives $w = z/(1 + 2\lambda)$, zero only if $z = 0$.



The geometry (Review Lec 3, Q12). By the soft-constraint theorem, ridge fits inside the disc $\|\theta\|_2^2 \le c$ and lasso inside the diamond $\|\theta\|_1 \le c$. The level sets of the training loss are ellipses around the unconstrained optimum; the solution is where the smallest ellipse touches the region. A disc has no corners, so the touching point generically has all coordinates non-zero: L_2 shrinks everything and keeps everything. The diamond has corners *on the axes*, and an expanding ellipse usually hits a corner first, where some coordinates are exactly 0: slide example, unconstrained $(0.6, 2.2)$, lasso $(0, 1.5)$, feature 1 dropped. L_1 is *embedded feature selection*, unstable under correlated features (it may pick either of two near-duplicates, where L_2 splits the weight evenly). Use L_1 when few features matter and you want to know which; L_2 when many contribute a little.

Early stopping as implicit regularisation (Review Lec 3, Q13)

Iterative optimisation produces $\hat{\theta}_0, \hat{\theta}_1, \ldots$ starting from an initial point. Early iterations learn strong recurring patterns; later ones fit exceptional observations or noise. **Early stopping** keeps the checkpoint with the smallest validation risk,

$$t^* = \arg\min_t \widehat{R}_{\text{val}}(\hat{\theta}_t), \qquad \hat{\theta}_{\text{ES}} = \hat{\theta}_{t^*}.$$

Stopping after t steps limits how far the parameters can travel from θ_0, so only part of the parameter space is reachable: a smaller effective family with the parameter count unchanged.

Training time is the capacity dial, and no penalty term is needed, hence *implicit*. Validation loss is noisy, so a **patience** p lets training continue p epochs without improvement before stopping, always returning the best checkpoint seen. Too small a patience stops on a random wobble (or before an epoch-wise second descent); too large wastes computation.

Tutorial drill

Review Lec 3, Q12, with numbers (Math set 3, Problem 3.5(b)). *Compare L_1 and L_2; why can L_1 produce sparse solutions? Orthonormal design, least-squares coefficients $z = (3, -0.5, 1.2, 0.1)$, $\lambda = 0.6$: compute both solutions and count non-zeros.*
Both shrink toward 0, trading variance for bias. L_2 per coordinate: $z/(1+2\lambda)$, zero only when $z = 0$. L_1: $\operatorname{sign}(z)\max(|z| - \lambda, 0)$, exactly zero whenever $|z| \le \lambda$; geometrically the diamond's corners sit on the axes and the loss ellipse hits a corner first, while the disc has no corners. Numbers: lasso subtracts 0.6 from each $|z_j|$ and floors at 0: (2.4, 0, 0.6, 0), two non-zeros; ridge divides by 2.2: (1.364, −0.227, 0.545, 0.045), four non-zeros. Consequence: L_1 selects features for free (unstable under correlation); L_2 keeps all.
Review Lec 3, Q13 (with Math set 3, Problem 3.6). *Why is early stopping implicit regularisation? What does patience do? Validation losses per epoch: .90, .70, .58, .52, .50, .51, .53, .52, .49, .47, .46, .48; where does training stop with $p = 2$ and $p = 4$?*
Stopping bounds the distance travelled from θ_0, so the reachable set is a nested family indexed by t: capacity control without a penalty. Best loss 0.50 at epoch 5; epochs 6–8 fail to improve, epoch 9 improves (0.49). $p = 2$: stops after epoch 7, returns epoch 5 (0.50). $p = 4$: survives the bump, runs to the 12-epoch limit, returns epoch 11 (0.46). The bump is an epoch-wise second descent: short patience stops inside it, long patience finds the lower minimum at the cost of 12 epochs instead of 7. Always return the best checkpoint, not the last.

5 Selecting Features and Configurations Without Fooling Yourself

Subset selection is combinatorial; the practical searches (Review Lec 3, Q14)

Feature selection changes the representation the learner sees (fewer irrelevant or redundant variables, nothing unavailable at decision time); it is a data-dependent model choice and is evaluated as one. With d features there are 2^d subsets (each feature in or out), so the ideal $S^* \in \arg\min_S \widehat{R}_{\text{val}}(\hat{f}_S)$ needs 2^d fits; for $d = 30$ that is $2^{30} = 1{,}073{,}741{,}824$. Hence:

Method	Strength	Limitation
Correlation filter: score $s_j = \lvert\widehat{\operatorname{Corr}}(X_j, Y)\rvert$, keep the top	very fast; model-independent	one variable at a time: misses nonlinear effects and interactions; ignores redundancy
Forward selection: from $\emptyset$, add the feature with largest validation gain	cheap; sees some joint effects	greedy: an early choice can block a better combination later
Backward elimination: from all features, remove the least useful	judges each feature jointly with all others	greedy; needs the full model fitted, costly in high dimension

Interaction counterexample: X_1, X_2 independent fair ± 1 coins, $Y = X_1 X_2$. Each has correlation 0 with Y, so a filter discards both; together they determine Y exactly.

Training, validation, test, and the overuse of validation (Review Lec 3, Q15)

Let h be an entire **configuration** (features, family, λ, k, threshold) from a candidate set $\mathcal{C}$. **Training** data fit the parameters *and* every data-dependent preprocessing step. **Validation** data compare configurations: $\hat{h} \in \arg\min_{h\in\mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h)$. **Test** data estimate the risk of the final frozen procedure, *once*. Validation is overused when many configurations are compared on it, or results are inspected repeatedly to guide the next design choice: the development process adapts to the validation set and its scores become optimistic.

The best of many validation scores is optimistic

If each $\widehat{R}_{\text{val}}(\hat{f}_h)$ is an unbiased estimate of $R(\hat{f}_h)$ (fitted without the validation data), then

$$\mathbb{E}\Big[\min_{h\in\mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h)\Big] \;\le\; \min_{h\in\mathcal{C}} \mathbb{E}[\widehat{R}_{\text{val}}(\hat{f}_h)] = \min_{h\in\mathcal{C}} R(\hat{f}_h),$$

strict as soon as the estimates are noisy and no single h wins in every sample. *Proof*: for every h_0, $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ pointwise; take expectations, then minimise the right side over h_0. Minima of noisy things are pulled downward ("winner's curse"). The test set protects against this *only* if its results never feed back into development; otherwise it has become another validation set.

K-fold cross-validation (Review Lec 3, Q16)

Partition the development data into K folds. For configuration h and fold j, fit $\hat{f}_h^{(-j)}$ on all folds except j and evaluate on fold j:

$$\widehat{R}_{\text{CV}}(h) = \frac{1}{K}\sum_{j=1}^{K} \widehat{R}_{\text{val}}^{(j)}(\hat{f}_h^{(-j)}), \qquad \hat{h} \in \arg\min_{h\in\mathcal{C}} \widehat{R}_{\text{CV}}(h).$$

Each observation validates once and trains $K-1$ times, at about K fits per configuration. **Preprocessing and feature selection are part of the fitted model**, so they are re-fitted inside every fold on its $K-1$ training folds only; otherwise the held-out fold has already shaped the representation used to predict it (leakage, optimistic $\widehat{R}_{\text{CV}}$). Random folds assume exchangeable observations; grouped or temporal dependence needs grouped or time-ordered folds (Section 6). After selection, refit the whole pipeline on all development data; CV does not replace the final test on untouched data.

Tutorial drill

Tutorial 1, Q5(d). *Two configurations have the same population error $R = 0.20$; validation errors $\widehat{R}_i = R+\varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0,\sigma^2)$ independent, $\sigma = 0.02$; the smaller is selected. Using* $\min(a,b) = \frac{a+b-|a-b|}{2}$ *and* $\mathbb{E}|\varepsilon_1-\varepsilon_2| = 2\sigma/\sqrt{\pi}$*, derive the expected reported error.*

$\mathbb{E}[\min(\widehat{R}_1,\widehat{R}_2)] = \frac{\mathbb{E}\widehat{R}_1+\mathbb{E}\widehat{R}_2}{2} - \frac{\mathbb{E}|\widehat{R}_1-\widehat{R}_2|}{2} = \frac{2R}{2} - \frac{2\sigma/\sqrt{\pi}}{2} = R - \frac{\sigma}{\sqrt{\pi}}$, since $\widehat{R}_1 - \widehat{R}_2 = \varepsilon_1 - \varepsilon_2$. Numerically $0.20-0.02/1.7725 = 0.20-0.0113 \approx 0.189$. Neither model is better, yet the reported score is 1.1 points below the truth: selecting the minimum picks whichever got the luckier noise. With three equal candidates ($\mathbb{E}\min(Z_1,Z_2,Z_3) = -3/(2\sqrt{\pi})$) it is $0.20 - 0.0169 = 0.183$: more candidates, more optimism. Consequence: the selected model must be re-measured on a test set that played no part in the choice.

Review Lec 3, Q16. *Explain K-fold CV. Why must preprocessing and feature selection be repeated inside each fold?*

K rotations of "fit on $K-1$ folds, score on the Kth", averaged; every example is validated exactly once. The scaler's $(\hat{\mu}_j, \hat{\sigma}_j)$, the imputer's fill values and the selector's chosen subset are *learned quantities*; fitting them on all development data lets the held-out fold influence the representation

it is judged on, so $\widehat{R}_{\mathrm{CV}}$ is optimistic (badly so when selecting among many features). Rule: cross-validate the whole pipeline, never the model alone.
New. *A team reports "validation accuracy* 0.97*, best of 120 configurations" as final performance. Name the error and the fix.*
0.97 is $\min_h$ of 120 noisy estimates, optimistic by the proposition above, and no untouched data remains. Fix: freeze $\hat{h}$, refit on all development data, evaluate once on a test set never used in the comparisons, and stop tuning after seeing it.

6 Honest Evaluation: Splits, Metrics, Leakage, Learning Curves

The split must match deployment. Ask first: what kind of *new* case will the model face? Then hold out exactly that kind of case.

Random, stratified, grouped, temporal (Review Lec 3, Q17)

Random: each row assigned to a subset approximately independently; credible only when deployment cases are like independent draws from the same population (quality inspection of items from a stable production line, each item photographed once). *Stratified*: random but preserving class proportions in every subset, which stabilises rare-class counts; it must not override dependence (records from one device must not be scattered to match proportions; stratification does *not* remove group leakage). *Grouped*: every group (user, device, patient, hospital, attack episode) lies entirely in one subset, so the model is scored on unseen groups (medical scans with several images per patient; "does the detector work on a hospital it has never seen"). *Temporal*: train on the past, validate and test on later data, preserving the direction of prediction and revealing drift (intrusion or fraud detection, churn next quarter, demand forecasting).

One intrusion dataset, three verdicts: flows from 50 devices over 6 months. A *random* split lets the model learn "device 17 is the compromised laptop": 99.5%; a *grouped* split holding out 10 devices forces it to recognise attack *behaviour*: 96%; a *temporal* split (train months 1–4, test 5–6, when a new attack tool appeared): 90%. Only the third number describes next month; the first answers a question nobody asked.

Accuracy under imbalance; the confusion-matrix metrics (Review Lec 3, Q18)

If a fraction π of the data is positive, "always predict negative" has accuracy $1-\pi$ and recall 0: with $\pi = 0.01$, 99% accuracy and no attack ever detected. With attacks as the positive class, count TP, FN, FP, TN and define

$$\text{Recall} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}, \quad \text{Precision} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}, \quad \text{Specificity} = \frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FP}}, \quad F_1 = \frac{2\,\mathrm{P}\,\mathrm{R}}{\mathrm{P}+\mathrm{R}},$$

and balanced accuracy $= (\text{Recall} + \text{Specificity})/2$. *Recall*: of the real attacks, what fraction did we catch? *Precision*: of the alarms we raised, what fraction were real? F_1: the harmonic mean, high only when both are high; it ignores TN and weighs the two errors equally. Fact: $\min(P,R) \le F_1 \le \frac{P+R}{2}$, since $\frac{P+R}{2} - F_1 = \frac{(P-R)^2}{2(P+R)} \ge 0$; F_1 is dragged toward the *worse* rate.

Quick numbers: 10,000 flows, 50 attacks, TP = 40, FN = 10, FP = 90, TN = 9860 give accuracy 0.990, recall 0.80, precision $40/130 = 0.31$, specificity 0.991, balanced accuracy 0.895, $F_1 = 2(0.31)(0.80)/1.11 = 0.44$ (far below the average 0.56). Accuracy says "99%"; precision says two alarms in three are false; which matters depends on whether an analyst must investigate each alarm.

Leakage, and fitting a scaler (Review Lec 3, Q19)

Leakage: development uses information unavailable when a new deployment decision is made. Forms: near-duplicates across subsets; the same device, session or patient in train and test; predictors created *after* the outcome ("ticket closed as intrusion", "number of analyst comments"); preprocessing fitted on validation or test rows; hyperparameters repeatedly chosen on test results. **Fitting a scaler** means estimating, from the training rows only,

$$\hat{\mu}_j = \frac{1}{N_{\text{tr}}} \sum_{i \in \text{tr}} x_j^{(i)}, \qquad \hat{\sigma}_j^2 = \frac{1}{N_{\text{tr}}} \sum_{i \in \text{tr}} \left(x_j^{(i)} - \hat{\mu}_j\right)^2,$$

then applying $z_j = (x_j - \hat{\mu}_j)/\hat{\sigma}_j$ *unchanged* to validation, test and deployment data. These are learned quantities, like parameters; estimating them on test rows lets those rows shape the representation they are judged on, and at deployment the statistics of future data do not exist anyway. Leaking: all data → estimate μ, σ → split. Valid: split → estimate on training → reuse. Same rule for imputation, feature selection, dimensionality reduction and resampling.

Learning curves (Review Lec 3, Q20)

A **learning curve** varies the training-set size N and plots training and validation error against it.

Observed pattern	Likely issue	Response
Both errors high, close, flattening as N grows	underfitting	improve the representation or increase capacity; more data will not help
Training low, validation much higher; gap closes slowly with N	high variance	regularise, reduce capacity, or collect representative data
Errors close and both still decreasing as N grows	data-limited	more representative examples are likely to help
Test performance degrades over time	distribution shift	update the data; time-aware evaluation protocol

Curves are diagnostic evidence, not proofs; read them with domain knowledge and error analysis.

Tutorial drill

Tutorial 1, Q4(a). *A team standardises all* 200,000 *flow records before splitting, randomly splits rows (one attack episode produces many near-duplicate flows), tries 100 configurations, selects the one with highest test accuracy and reports that accuracy. Identify every mechanism that makes the report too optimistic.*

(1) Preprocessing leakage: $\hat{\mu}, \hat{\sigma}$ computed with test rows. (2) Group leakage: near-duplicates of one episode land on both sides, so the test set rewards recognising seen episodes, not detecting new ones; group by episode (or split temporally for drift). (3) Selection on the test set: best of 100 noisy scores, $\mathbb{E}[\min_h \widehat{R}] \leq \min_h R$ (Section 5). (4) No untouched data remains for the chosen model. (5) Accuracy under imbalance is high even when most attacks are missed. Fix: split first (grouped or temporal), fit preprocessing on training rows, select on validation or grouped CV, test once.

Tutorial 1, Q4(c)–(d). *Test set:* 10,000 *flows,* 100 *attacks; TP* = 70, *FP* = 180, *FN* = 30, *TN* = 9720. *Compute accuracy, precision, recall, F_1; compare with "everything benign".*

Accuracy $(70 + 9720)/10000 = 0.979$; precision $70/250 = 0.28$; recall $70/100 = 0.70$; $F_1 = 2(0.28)(0.70)/0.98 = 0.40$. "Everything benign": TP = 0, FN = 100, TN = 9900: accuracy 0.99, *higher* than the real classifier, with recall 0 and precision undefined (0/0). Accuracy is a weighted average dominated by the 99% benign flows, so it rewards the useless rule; the real detector catches 70% of attacks at about 2.6 false alarms per true detection, a trade-off accuracy cannot even express.

Review Lec 3, Q19 (with Math set 3, Problem 3.7). *What does it mean to fit a scaler, and why on training data only? Training values* $2, 4, 6, 8$*; test value* 20*; flag* $|z| > 3$.
Fit = estimate $\hat{\mu}, \hat{\sigma}$. Training only: $\hat{\mu} = 5$, $\hat{\sigma}^2 = (9 + 1 + 1 + 9)/4 = 5$, $\hat{\sigma} = 2.236$, so $z = (20 - 5)/2.236 = 6.71 > 3$: flagged. Leaky fit on all five values: $\hat{\mu} = 8$, $\hat{\sigma}^2 = (36 + 16 + 4 + 0 + 144)/5 = 40$, $\hat{\sigma} = 6.32$, $z = (20 - 8)/6.32 = 1.90 < 3$: *not* flagged. The test point inflated the scale used to judge it and hid itself. Leakage does not merely flatter scores; it changes decisions.
Review Lec 3, Q20. *How do learning curves distinguish underfitting, high variance and a data-limited regime?*
Read the *level* of the two curves and the *gap* between them as N grows: both high and flat with a small gap is underfitting (data will not help); training far below validation with a slowly closing gap is high variance (more data shrinks variance, never bias); a small gap with both still falling is data-limited. The curve says which lever to pull before paying for data.

The mistakes that cost points

- Diagnosing with one number. Underfitting and overfitting are read from the *pair* (training, validation) and their gap; "zero training error" is a warning, not an achievement, since nested capacity guarantees it. And the kNN dial is $1/k$: *small* k is high capacity.
- Writing bias and variance as properties of one fitted model. Both are expectations over *repeated training sets* at a fixed x; the noise $\sigma^2(x)$ is a third term that even f^* pays.
- Saying L_1 "shrinks more". The point is *exact zeros*: soft thresholding $\operatorname{sign}(z)\max(|z| - \lambda, 0)$ versus ridge's $z/(1 + 2\lambda)$; the diamond's corners sit on the axes.
- Treating the best validation score as the final result, or fitting the scaler/selector before splitting. Both are leakage: the minimum of noisy estimates is optimistic, and stratification does not cure group dependence.

Chapter 3 in five lines

1. $\widehat{R}_{\mathcal{D}}$ is observed, R is wanted, $G = R - \widehat{R}$ compares them for one fitted model; the data choose and then grade the model, so $\mathbb{E}[G] \geq 0$. Inductive bias (data, $\mathcal{H}$, regularisation, optimiser) decides what happens off the sample.
2. Nested families: minimum training error can only fall with capacity, so capacity is chosen on validation; underfitting is both errors high, overfitting a large gap; the kNN dial is $1/k$; capacity is relative to N.
3. At a fixed x, over repeated training sets: squared error $= \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x)$ (add and subtract two means); noise is irreducible; double descent can add a second fall past the interpolation threshold.
4. A penalty $\lambda\Omega(\theta)$ is a soft constraint $\Omega \leq c$: smaller effective family, same parameter count. L_2 shrinks $(z/(1 + 2\lambda))$, L_1 soft-thresholds to exact zeros (diamond corners). Early stopping is the same control over training time; patience rides out noise.
5. Subset selection is 2^d (filters, forward, backward: each blind to something). The minimum validation score is optimistic ($R - \sigma/\sqrt{\pi}$ for two equal models); K-fold CV refits preprocessing inside each fold; the test set is used once. Split as deployment demands (random, grouped, temporal); under imbalance report recall, precision, F_1; fit scalers on training rows only; read learning curves by level and gap.